CLOUD & VIRTUALISATION | TEK-UP UNIVERSITY

Lab 05

Dockerfile and registry: containerising a website and publishing it

Student
Assil Mhadhbi

Class
Ing1, Groupe C (evening course)

Date
31 October 2026

About this report

In this lab I wrote a Dockerfile for the home page of the TEK-UP Cloud club, a small static website served by nginx. I built two versions of it (1.0 and 1.1), published both in a private registry that runs in a container and keeps its data in a volume, and redeployed the site from that registry only. I moved from 1.0 to 1.1 and back by changing nothing but the tag, and as a bonus I pushed version 1.1 to my Docker Hub account.

Three things are worth knowing before the details. First, I used the Alpine variant of nginx with an exact version (1.30.6), which I read from the image itself instead of guessing. Second, I typed several files by hand and made mistakes in them, so some parts of the report are in boxes called Problem I ran into; each mistake taught me something about how Docker and nginx read what I give them. Third, the VM is behind VirtualBox's NAT network, so to open the site from my Windows browser I had to add a port forwarding rule.

The report follows the order of the handout and every item of the two checklists is done, including the bonus. The Docker Hub access token does not appear anywhere in this report. The answers to the five questions of the handout are in section 6.

1. My setup

Item	What I used
Virtual machine	Red Hat Enterprise Linux in VirtualBox (NAT network, address 10.0.2.15), user darkdiva
Docker	Docker Engine – Community 29.9.1, with BuildKit as the default builder
Base image	nginx:1.30.6-alpine (nginx 1.30.6 on Alpine)
Working folder	~/lab05, containing the site, the nginx configuration, the Dockerfile and the .dockerignore
Ports on the VM	8080 for the site 1.0, 8081 for the redeployed site, 8082 for a quick test of 1.1, 5000 for the registry
Registry	The registry:2 image in a container named registry, with the named volume registry-data
Docker Hub	Account darkdiva (bonus)

2. The site to containerise

I created the folder tree of the handout in `~/lab05`:

```
lab05/
├── site/
│   ├── index.html
│   └── 404.html
└── nginx/
    └── default.conf
```

`index.html` is the home page (“Club Cloud TEK-UP”), `404.html` is my own error page, and `default.conf` is the nginx server configuration that goes into the image. It listens on port 80, serves `/usr/share/nginx/html`, and sends missing pages to `/404.html`:

```
server {
    listen       80;
    server_name  localhost;
    root         /usr/share/nginx/html;
    index        index.html;
    error_page   404 /404.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

```
darkdiva@localhost:~$ mkdir -p ~/lab05/site ~/lab05/nginx
darkdiva@localhost:~$ cd ~/lab05
```

Figure 1. Creating the folders site and nginx in ~/lab05.

```
darkdiva@localhost:~/lab05$ find ~/lab05 -type f
/home/darkdiva/lab05/site/index.html
/home/darkdiva/lab05/site/404.html
/home/darkdiva/lab05/nginx/default.conf
darkdiva@localhost:~/lab05$
```

Figure 2. The three files of the site in place.

3. Part 1: writing the Dockerfile

3.1 Studying the base image. The handout asks for an official nginx image in the Alpine variant with a fixed version. I first pulled `nginx:stable-alpine` only to look at it, and `nginx -v` told me that it contains nginx **1.30.6**. I then pulled `nginx:1.30.6-alpine`: the digest is the same, so it is the same image, and this exact tag is the one I pin in the Dockerfile.

```
darkdiva@localhost:~/lab05$ docker pull nginx:stable-alpine
stable-alpine: Pulling from library/nginx
5b8cb54cfa40: Pull complete
885725a63a7c: Pull complete
7b229517ed22: Pull complete
9f0afb44f10e: Pull complete
f19a2d3b6951: Pull complete
81b64ae234ef: Pull complete
e125989d16f0: Pull complete
bb658a254196: Pull complete
91e3d1cf855d: Download complete
4675ff125325: Download complete
Digest: sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d6094767ac30e97949
Status: Downloaded newer image for nginx:stable-alpine
docker.io/library/nginx:stable-alpine
darkdiva@localhost:~/lab05$ docker run --rm nginx:stable-alpine nginx -v
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
nginx version: nginx/1.30.6
darkdiva@localhost:~/lab05$
```

Figure 3. Pulling nginx:stable-alpine and reading the version with nginx -v (1.30.6).

`docker inspect` shows what the base image already defines: the entrypoint `/docker-entrypoint.sh`, the command `nginx -g daemon off;` and the port 80. The listing of the image confirms where nginx looks for its files: the configuration is `/etc/nginx/conf.d/default.conf` and the pages are in `/usr/share/nginx/html`. The Alpine image has no `curl`, but `wget` is at `/usr/bin/wget`, which I use for the health check.

```
darkdiva@localhost:~/lab05$ docker inspect nginx:stable-alpine --format 'Entypoint={{.Config.Entrypoint}} Cmd={{.Conf
ig.Cmd}} Port{{.Config.ExposedPorts}}'
Entypoint=[/docker-entrypoint.sh] Cmd=[nginx -g daemon off;] Portmap[80/tcp:{}]
```

Figure 4. docker inspect: Entrypoint, Cmd and the exposed port of the base image.

```
darkdiva@localhost:~/lab05$ docker run --rm nginx:stable-alpine ls /etc/nginx/conf.d /usr/share/nginx/html
/etc/nginx/conf.d:
default.conf

/usr/share/nginx/html:
50x.html
index.html
darkdiva@localhost:~/lab05$ docker run --rm nginx:stable-alpine which wget
/usr/bin/wget
darkdiva@localhost:~/lab05$ docker pull nginx:1.30.6-alpine
1.30.6-alpine: Pulling from library/nginx
Digest: sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d6094767ac30e97949
Status: Downloaded newer image for nginx:1.30.6-alpine
docker.io/library/nginx:1.30.6-alpine
darkdiva@localhost:~/lab05$
```

Figure 5. Configuration and pages folders, location of wget, and the pull of the pinned tag.

3.2 The Dockerfile. Every instruction has a one-line comment, as the handout asks. I placed the instructions that change often (the site pages and the version label) at the end, so that the layers above them stay in the cache when I build version 1.1.

```
#Image officielle nginx, variante Alpine, version fixée (pas latest)
FROM nginx:1.30.6-alpine
#Etiquette 1 : Auteur de l'image
LABEL org.opencontainers.image.authors="Assil Mhadhbi"
#Documente le port d'écoute de nginx (ne le publie pas)
EXPOSE 80
#Test de santé: Wget (présent sur Alpine, contrairement à curl) demande la
page d'accueil
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
CMD wget -q -O /dev/null http://127.0.0.1 || exit 1
#Remplace la configuration par défaut de nginx par la mienne
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
#Copie les pages du site là où nginx les cherche
COPY site/ /usr/share/nginx/html/
#Etiquette 2 : version du site (en dernier, pour garder le cache au-dessus)
LABEL org.opencontainers.image.version="1.0"
```

There is no `CMD`, because the base image already starts nginx (see the first question in section 6). The health check uses `127.0.0.1` and not `localhost`, because my `default.conf` only listens on IPv4.

3.3 The .dockerignore. It keeps out of the image what does not belong there. To check it, I created a `.git` folder and a file `notes-perso.txt`, and I also copied the notes into `site/` so that the exclusion is tested on a file that a `COPY site/` would really take.

```
# Données de gestion de versions
.git
**/.git
# Notes personnelles
notes-perso.txt
**/notes-perso.txt
# Fichiers de build qui n'ont rien à faire dans l'image
Dockerfile
.dockerignore
```

```
darkdiva@localhost:~/lab05$ mkdir -p .git
darkdiva@localhost:~/lab05$ echo "fake git data" > .git/config
darkdiva@localhost:~/lab05$ echo "mes notes privées" > notes-perso.txt
darkdiva@localhost:~/lab05$ cp notes-perso.txt site/notes-perso.txt
darkdiva@localhost:~/lab05$
```

Figure 6. Creating the files that must stay out of the image.

```
darkdiva@localhost:~/lab05$ nano Dockerfile
darkdiva@localhost:~/lab05$ nano .dockerignore
darkdiva@localhost:~/lab05$ cat .dockerignore
# Données de gestion de versions
.git
**/.git
# Notes personnelles
notes-perso.txt
**/notes-perso.txt
#Fichiers de build qui n'ont rien à faire dans l'image
Dockerfile
.dockerignore
darkdiva@localhost:~/lab05$ cat Dockerfile
#Image officielle nginx, variante Alpine, version fixée (pas latest)
FROM nginx:1.30.6-alpine
#Etiquette 1 : Auteur de l'image
LABEL org.opencontainers.image.authors="Assil Mhadhbi"
#Documente le port d'écoute de nginx (ne le publie pas)
EXPOSE 80
#Test de santé: Wget (présent sur Alpine, contrairement à curl) demande la page d'acceuil
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
CMD wget -q -O /dev/null http://127.0.0.1 || exit 1
#Remplace la configuration par défaut de nginx par la mienne
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
#Copie les pages du site là ou nginx les cherche
COPY site/ /usr/share/nginx/html/
#Etiquette 2 : version du site (en dernier, pour garder le cache au-dessus)
LABEL org.opencontainers.image.version="1.0"
darkdiva@localhost:~/lab05$
```

Figure 7. The .dockerignore and the Dockerfile as they are in my folder.

```
/home/darkdiva/lab05/Dockerfile
/home/darkdiva/lab05/.dockerignore
/home/darkdiva/lab05/.git/config
/home/darkdiva/lab05/nginx/default.conf
/home/darkdiva/lab05/notes-perso.txt
/home/darkdiva/lab05/site/404.html
/home/darkdiva/lab05/site/index.html
/home/darkdiva/lab05/site/notes-perso.txt
darkdiva@localhost:~/lab05$
```

Figure 8. All the files in ~/lab05, including the .git folder and the notes.

3.4 First build. `docker build -t monsite:1.0 .` took 7.8 seconds. Docker prints a warning that git is not installed on the VM; it is harmless, because it only concerns a commit number that BuildKit tried to record from my fake `.git` folder.

```
darkdiva@localhost:~/lab05$ docker build -t monsite:1.0 .
[+] Building 7.8s (8/8) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile                                                                      0.2s
 => => transferring dockerfile: 889B                                                                                      0.0s
 => [internal] load metadata for docker.io/library/nginx:1.30.6-alpine                                                    0.1s
 => [internal] load .dockerignore                                                                                         0.1s
 => => transferring context: 286B                                                                                         0.0s
 => [1/3] FROM docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d609            4.3s
 => => resolve docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d609            0.3s
 => => extracting sha256:7b229517ed222597277036d94892173207d01a731ab3a88eeb1de2e963a2c788                                 0.6s
 => => extracting sha256:9f0afb44f10e826253b342c0265e188067c2f0a30c782a52e329928a763adc60                                 0.6s
 => => extracting sha256:f19a2d3b695178ba0e57d473535f74f5d934f45e191de8334e730f83d2c62704                                 0.1s
 => => extracting sha256:885725a63a7cf1d4d5aac105944b35ece2b69a8c2cc6f7040f04727b7b8a63d2                                 0.1s
 => => extracting sha256:e125989d16f03d1e00b60afdd9abe46ed1437ed63b73926d036a5537f6be770a                                 0.1s
 => => extracting sha256:81b64ae234ef95f3cfc19b7f6c45a511eccbefa6148889ee33ed43c6c3f39939                                 0.1s
 => => extracting sha256:bb658a2541968d2b90a8ee165ad08b7f71055c7f7832bdff98e2de2d8b90cf14                                 0.1s
 => => extracting sha256:5b8cb54cfa408030aaa43d7bffa9141e0ae6f8a479df13c7aa152100cca358fb                                 1.2s
 => [internal] load build context                                                                                         0.3s
 => => transferring context: 1.05kB                                                                                       0.1s
 => [2/3] COPY nginx/default.conf /etc/nginx/conf.d/default.conf                                                          0.6s
 => [3/3] COPY site/ /usr/share/nginx/html/                                                                               0.2s
 => exporting to image                                                                                                    1.5s
 => => exporting layers                                                                                                   0.7s
 => => exporting manifest sha256:6a0c9752d73343187a20406e5ab4d02a59cfd33c2ed2f36b91d6d21d8a44a67a                         0.1s
 => => exporting config sha256:d8a3e18b5e1575ea4bf1558050e3c01e8b672f04c81ac716b83d8e1e2d799946                           0.1s
 => => exporting attestation manifest sha256:ecf1295d4379d55e486a5487d2a7d4e5a02ead09292246d1b434fdfa67a460ac             0.1s
 => => exporting manifest list sha256:e423adfdd0b21a30dd919e0aee203b478e07c449a34f5342d45117abfb6c29c7                    0.1s
 => => naming to docker.io/library/monsite:1.0                                                                            0.0s
 => => unpacking to docker.io/library/monsite:1.0                                                                         0.3s
WARNING: current commit information was not captured by the build: git was not found in the system: exec: "git": exec
utable file not found in $PATH
darkdiva@localhost:~/lab05$
```

Figure 9. Build of monsite:1.0.

Two checks prove that the image is what I wanted. The list of `/usr/share/nginx/html` inside the image shows `404.html`, `50x.html` (from the base image) and `index.html`, with no `notes-perso.txt` and no `.git`, so the `.dockerignore` works. The labels show my author and the version 1.0 (the `maintainer` label is inherited from the base image).

```
darkdiva@localhost:~/lab05$ docker run --rm monsite:1.0 ls -a /usr/share/nginx/html
.
..
404.html
50x.html
index.html
```

Figure 10. Content of the pages folder in the image: no notes-perso.txt.

```
darkdiva@localhost:~/lab05$ docker inspect monsite:1.0 --format '{{json .Config.Labels}}'
{"maintainer":"NGINX Docker Maintainers <docker-maint@nginx.com>","org.opencontainers.image.authors":"Assil Mhadhbi",
"org.opencontainers.image.version":"1.0"}
darkdiva@localhost:~/lab05$
```

Figure 11. The labels of monsite:1.0.

Problem I ran into

My first container did not stay up: `docker ps` showed nothing, `docker ps -a` showed `Exited (1)`, and `docker logs web` ended with `directive "error_page" is not terminated by ";"`. I had typed `default.conf` by hand with three mistakes: a missing `;` at the end of the `error_page` line, `try_filrs` instead of `try_files`, and `404/404.html` without a space.

After I added the semicolon, nginx answered `invalid number of arguments in "error_page" directive`. `cat -A`, which shows the hidden characters and the end of each line, made the missing space easy to see. I also learned that running `nginx -t` on the old image proves nothing: the file is copied into the image at build time, so every fix needs a new `docker build` before the test.

```
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
darkdiva@localhost:~/lab05$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED              STATUS                          PORTS
NAMES
e713e88ae306   monsite:1.0   "/docker-entrypoint.…"   About a minute ago   Exited (1) About a minute ago
web
f7795f10d0ab   hello-world   "/hello"                 2 weeks ago          Exited (0) 2 weeks ago
naughty_borg
darkdiva@localhost:~/lab05$ docker logs web
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/31 17:08:57 [emerg] 1#1: directive "error_page" is not terminated by ";" in /etc/nginx/conf.d/default.conf:8
nginx: [emerg] directive "error_page" is not terminated by ";" in /etc/nginx/conf.d/default.conf:8
darkdiva@localhost:~/lab05$ nano nginx/dafault.conf
darkdiva@localhost:~/lab05$ nano nginx/default.conf
darkdiva@localhost:~/lab05$ docker rm web
web
darkdiva@localhost:~/lab05$ docker build -t mon-site:1.0 .
[+] Building 2.8s (8/8) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                        0.0s
 => => transferring dockerfile: 889B                                                                        0.0s
 => [internal] load metadata for docker.io/library/nginx:1.30.6-alpine                                      0.1s
 => [internal] load .dockerignore                                                                           0.0s
 => => transferring context: 286B                                                                           0.0s
```

Figure 12. The container exited with code 1 and the nginx error in docker logs.

```
darkdiva@localhost:~/lab05$ docker run --rm monsite:1.0 nginx -t
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/31 17:27:44 [emerg] 1#1: invalid number of arguments in "error_page" directive in /etc/nginx/conf.d/default.c
onf:6
nginx: [emerg] invalid number of arguments in "error_page" directive in /etc/nginx/conf.d/default.conf:6
nginx: configuration file /etc/nginx/nginx.conf test failed
darkdiva@localhost:~/lab05$ cat -A nginx/default.conf
server {$
listen      80;$
server_name localhost;$
root /usr/share/nginx/html;$
index index.html;$
error_page  404/404.html;$
location / {$
try_files $uri $uri/ =404;$
}$
}$
darkdiva@localhost:~/lab05$ cat >nginx/default.conf << 'EOF'
server {
listen      80;
server_name localhost;
root /usr/share/nginx/html;
index index.html;
error_page  404 /404.html;
location / {
try_files $uri $uri/ =404;
}
}
EOF
darkdiva@localhost:~/lab05$ docker rm -f web
darkdiva@localhost:~/lab05$ docker build -t monsite:1.0 .
[+] Building 2.9s (8/8) FINISHED                                                                    docker:default
```

Figure 13. The failed test of the configuration and cat -A showing 404/404.html without a space.

3.5 Running the site and checking the health. After the corrections, I rebuilt the image and tested the configuration of the new image with `nginx -t` ("syntax is ok", "test is successful"). Then I started the container in the background and published it on port 8080 of the VM with `docker run -d --name web -p 8080:80 monsite:1.0`. `docker ps` shows the status `Up 5 seconds (healthy)`: the HEALTHCHECK works.

```
darkdiva@localhost:~/lab05$ docker build -t monsite:1.0 .
[+] Building 2.9s (8/8) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                        0.1s
 => => transferring dockerfile: 889B                                                                                        0.0s
 => [internal] load metadata for docker.io/library/nginx:1.30.6-alpine                                                      0.1s
 => [internal] load .dockerignore                                                                                           0.0s
 => => transferring context: 286B                                                                                           0.0s
 => CACHED [1/3] FROM docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe              0.2s
 => => resolve docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d609              0.1s
 => [internal] load build context                                                                                           0.1s
 => => transferring context: 628B                                                                                           0.0s
 => [2/3] COPY nginx/default.conf /etc/nginx/conf.d/default.conf                                                            0.1s
 => [3/3] COPY site/ /usr/share/nginx/html/                                                                                 0.2s
 => exporting to image                                                                                                      1.6s
 => => exporting layers                                                                                                     0.7s
 => => exporting manifest sha256:f4b671d52e529212e7676f6812a336cd4c88c5fd208683772da7e27d8f0abb93                           0.1s
 => => exporting config sha256:587338caca953ec2ee694cb69ee4974bc40d4a9110351e5132d2f619b8f64855                             0.1s
 => => exporting attestation manifest sha256:e753f9368c38c095d08b5ae9f60b548b9ff0800ce410fac15e065ba31068c407               0.1s
 => => exporting manifest list sha256:2a2565c044ee693d923024a7a09c7c7d28e167302982e837f7371d0525820ad8                      0.1s
 => => naming to docker.io/library/monsite:1.0                                                                              0.0s
 => => unpacking to docker.io/library/monsite:1.0                                                                           0.2s
WARNING: current commit information was not captured by the build: git was not found in the system: exec: "git": exec
utable file not found in $PATH
darkdiva@localhost:~/lab05$ docker run --rm monsite:1.0 nginx -t
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
darkdiva@localhost:~/lab05$ docker run -d --name web -p 8080:80 monsite:1.0
71b75cfd0d6ccf506e0b903c5ef2b6917b079390d5ed3d2a8c1da04515359074
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                   PORTS
                  NAMES
71b75cfd0d6c   monsite:1.0   "/docker-entrypoint.…"   5 seconds ago   Up 5 seconds (healthy)   0.0.0.0:8080->80/tcp,
[::]:8080->80/tcp   web
darkdiva@localhost:~/lab05$
```

Figure 14. Rebuild, nginx -t and docker ps showing the container healthy on port 8080.

3.6 The pages and the logs. `curl http://localhost:8080` returns the home page, and `curl -i http://localhost:8080/inconnu` returns `404 Not Found` with **my** page ("Cette page n'existe pas au club Cloud") and not the default page of nginx. The headers show `Server: nginx/1.30.6`.

```
darkdiva@localhost:~/lab05$ curl http://localhost:8080
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="utf-8">
<title>Club Cloud TEK-UP</title>
</head>
<body>
<h1>Club Cloud TEK-UP</h1>
<p>Version 1.0 - service par nginx dans un conteneur.</p>
</body>
</html>
darkdiva@localhost:~/lab05$ curl -i http://localhost:8080/inconnu
HTTP/1.1 404 Not Found
Server: nginx/1.30.6
Date: Sat, 31 Oct 2026 17:39:11 GMT
Content-Type: text/html
Content-Length: 174
Connection: keep-alive
ETag: "6ae61397-ae"

<!DOCTYPE html>
<html lNG="FR">
<head><meta charset="utf-8"><title>Page introuvable</title></head>
<body><h1> 404 - Cette page n'existe pas au club Cloud</h1></body>
</html>
darkdiva@localhost:~/lab05$ docker logs web
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/31 17:32:17 [notice] 1#1: using the "epoll" event method
2026/10/31 17:32:17 [notice] 1#1: nginx/1.30.6
2026/10/31 17:32:17 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/10/31 17:32:17 [notice] 1#1: OS: Linux 6.12.0-211.65.1.el10_2.x86_64
2026/10/31 17:32:17 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/10/31 17:32:17 [notice] 1#1: start worker processes
2026/10/31 17:32:17 [notice] 1#1: start worker process 28
```

Figure 15. The home page, and the custom 404 page for an address that does not exist.

The logs show the requests. The lines from `127.0.0.1` with the agent `Wget` are the health check, every 10 seconds. The line from `172.17.0.1` with the agent `curl` is my own request through the published port.

```
2026/10/31 17:32:17 [notice] 1#1: start worker processes
2026/10/31 17:32:17 [notice] 1#1: start worker process 28
2026/10/31 17:32:17 [notice] 1#1: start worker process 29
127.0.0.1 - - [31/Oct/2026:17:32:21 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:32:31 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:32:41 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:32:51 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:01 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:12 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:22 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:32 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:42 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:33:52 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:02 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:13 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:23 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:33 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:43 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:34:53 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:03 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:14 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:24 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:34 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:44 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:35:54 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:04 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:14 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:25 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:35 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:45 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:36:55 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:05 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:15 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:25 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:36 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:46 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:37:56 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:38:06 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:38:16 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:38:26 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:38:37 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
127.0.0.1 - - [31/Oct/2026:17:38:47 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
172.17.0.1 - - [31/Oct/2026:17:38:49 +0000] "GET / HTTP/1.1" 200 213 "-" "curl/8.12.1" "-"
127.0.0.1 - - [31/Oct/2026:17:38:57 +0000] "GET / HTTP/1.1" 200 213 "-" "Wget" "-"
```

Figure 16. docker logs web: the health check requests and my own request.

3.7 Opening the site from Windows. The handout suggests looking at the site from the browser of the PC. My VM has the address `10.0.2.15`, which is VirtualBox's NAT address, so it is private to the VM. I opened port 8080 in firewalld and added a port forwarding rule in VirtualBox.

> **Problem I ran into**
>
> My first attempt in Chrome on Windows gave `ERR_CONNECTION_REFUSED`. The site worked on the VM (the `curl` proved it), but on a NAT network `localhost` on the PC is the PC itself, not the VM. I added a rule in the VM settings (Network, Advanced, Port Forwarding): TCP, host 127.0.0.1 port 8080, guest 10.0.2.15 port 8080. After that the page loaded.

```
darkdiva@localhost:~/lab05$ ip a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host noprefixroute
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
    inet6 fd17:625c:f037:2:a00:27ff:fe5a:f711/64 scope global dynamic noprefixroute
    inet6 fe80::a00:27ff:fe5a:f711/64 scope link noprefixroute
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
    inet6 fe80::502c:beff:fe03:9133/64 scope link proto kernel_ll
    inet6 fe80::68a1:a7ff:fe3c:4399/64 scope link proto kernel_ll
darkdiva@localhost:~/lab05$ sudo firewall-cmd --list-ports
[sudo] password for darkdiva:

darkdiva@localhost:~/lab05$ sudo firewall-cmd --list-ports

darkdiva@localhost:~/lab05$ sudo firewall-cmd --add-port=8080/tcp
success
darkdiva@localhost:~/lab05$
```

Figure 17. The address of the VM and the opening of port 8080 in firewalld.

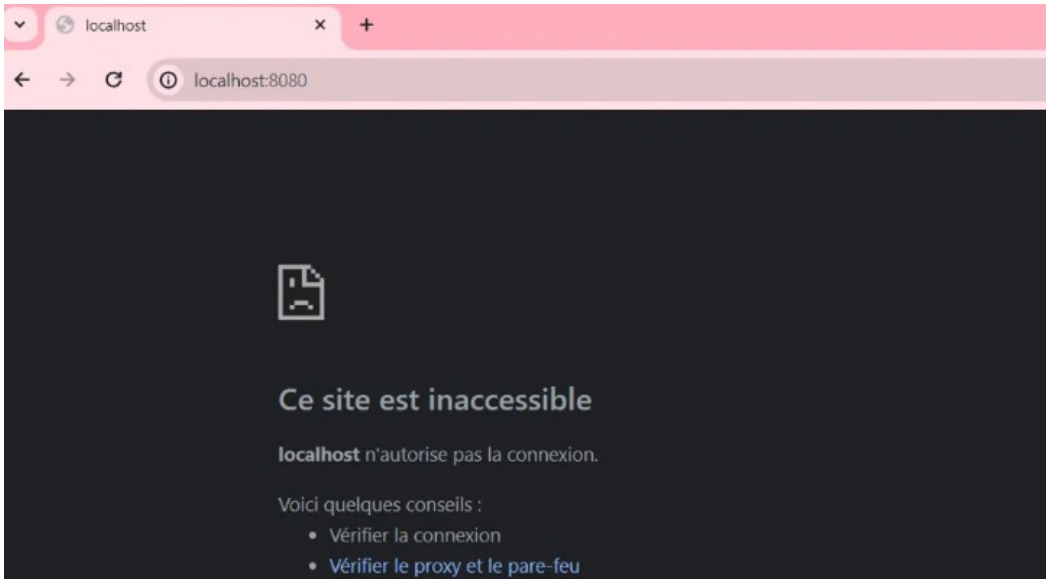


Figure 18. Chrome before the port forwarding rule: connection refused.

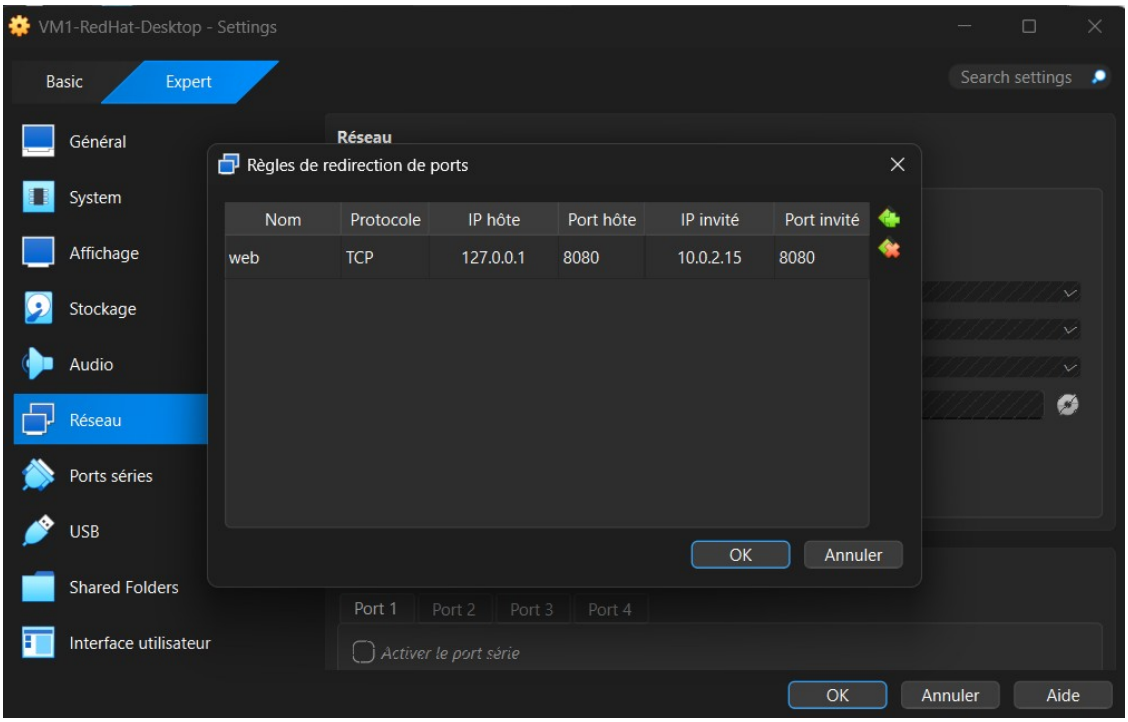


Figure 19. The port forwarding rule in the VirtualBox settings.



Figure 20. The site displayed in the browser of my PC.

3.8 Version 1.1. I changed the text of `index.html` to "Version 1.1", corrected a typo in the `lang` attribute of `404.html`, and changed the version label in the Dockerfile to 1.1. Then I built `monsite:1.1`.

```
darkdiva@localhost:~/lab05$ nano Dockerfile
darkdiva@localhost:~/lab05$ cat Dockerfile
#Image officielle nginx, variante Alpine, version fixée (pas latest)
FROM nginx:1.30.6-alpine
#Etiquette 1 : Auteur de l'image
LABEL org.opencontainers.image.authors="Assil Mhadhbi"
#Documente le port d'écoute de nginx (ne le publie pas)
EXPOSE 80
#Test de santé: Wget (présent sur Alpine, contrairement à curl) demande la page d'acceuil
HEALTHCHECK --interval=10s --timeout=3s --start-period=5s --retries=3 \
CMD wget -q -O /dev/null http://127.0.0.1 || exit 1
#Remplace la configuration par défaut de nginx par la mienne
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
#Copie les pages du site là ou nginx les cherche
COPY site/ /usr/share/nginx/html/
#Etiquette 2 : version du site (en dernier, pour garder le cache au-dessus)
LABEL org.opencontainers.image.version="1.1"
darkdiva@localhost:~/lab05$ cat ~/site/index.html
cat: /home/darkdiva/site/index.html: No such file or directory
```

Figure 21. The Dockerfile with the label 1.1.

```
darkdiva@localhost:~/lab05/site$ cat index.html
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="utf-8">
<title>Club Cloud TEK-UP</title>
</head>
<body>
<h1>Club Cloud TEK-UP</h1>
<p>Version 1.1 - service par nginx dans un conteneur.</p>
</body>
</html>
darkdiva@localhost:~/lab05/site$ cat 404.html
<!DOCTYPE html>
<html lang="FR">
<head><meta charset="utf-8"><title>Page introuvable</title></head>
<body><h1> 404 - Cette page n'existe pas au club Cloud</h1></body>
</html>
darkdiva@localhost:~/lab05/site$
```

Figure 22. index.html now says Version 1.1; 404.html with the corrected lang attribute.

> **Problem I ran into**
>
> My first `docker build -t monsite:1.1 .` failed with `open Dockerfile: no such file or directory`. My prompt showed `~/lab05/site`: I was inside the `site` folder, and the Dockerfile is one level up. The final dot is the build context, so Docker looked for a `Dockerfile` in the folder I was in. After `cd ~/lab05` the build worked.

```
darkdiva@localhost:~/lab05/site$ docker build -t monsite:1.1 .
[+] Building 0.4s (1/1) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                         0.1s
 => => transferring dockerfile: 2B                                                                           0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory
darkdiva@localhost:~/lab05/site$ S
```

Figure 23. The build run from the wrong folder.

In the build output, the base image is reused without any download or extraction, `[2/3] COPY nginx/default.conf` is marked **CACHED**, and only `[3/3] COPY site/` runs again, because the site changed. The LABEL instructions only change metadata and add no step. `docker images monsite` lists both versions. I then started 1.1 on port 8082 for a quick test: `curl` shows "Version 1.1" and an unknown address still returns 404.

```
darkdiva@localhost:~/lab05$ docker build -t monsite:1.1 .
[+] Building 2.1s (8/8) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                         0.0s
 => => transferring dockerfile: 889B                                                                         0.0s
 => [internal] load metadata for docker.io/library/nginx:1.30.6-alpine                                       0.1s
 => [internal] load .dockerignore                                                                            0.0s
 => => transferring context: 286B                                                                            0.0s
 => [1/3] FROM docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d609  0.2s
 => => resolve docker.io/library/nginx:1.30.6-alpine@sha256:0e52bd054126cdc040b1276c0af347aeb02a41b9cfe305d609  0.1s
 => [internal] load build context                                                                            0.1s
 => => transferring context: 862B                                                                            0.0s
 => CACHED [2/3] COPY nginx/default.conf /etc/nginx/conf.d/default.conf                                      0.0s
 => [3/3] COPY site/ /usr/share/nginx/html/                                                                  0.2s
 => exporting to image                                                                                       1.0s
 => => exporting layers                                                                                      0.4s
 => => exporting manifest sha256:4f91fd62a866f870309ad81843fe37e92eac80ffb70d07698b0aba36e9a4b25f            0.1s
 => => exporting config sha256:0bcf119e7cd81afec216130d3190ccb45115241ba7d6dc36d7b0e792b919eb64              0.1s
 => => exporting attestation manifest sha256:fad36bbaa0d227a74f75d7f9cb5daa0c0148abfbda1c3ead353316202fe5cae1  0.1s
 => => exporting manifest list sha256:765f8c3bb1852b6d71d065833cce858fd0afa0d5a9cf9f1c6f8fa4ba8e8de905       0.1s
 => => naming to docker.io/library/monsite:1.1                                                               0.0s
 => => unpacking to docker.io/library/monsite:1.1                                                            0.1s
WARNING: current commit information was not captured by the build: git was not found in the system: exec: "git": exec
utable file not found in $PATH
darkdiva@localhost:~/lab05$ docker images monsite
                                                                                  i Info →  U  In Use
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
monsite:1.0    2a2565c044ee       91.3MB         26.1MB    U
monsite:1.1    765f8c3bb185       91.3MB         26.1MB
darkdiva@localhost:~/lab05$ docker run -d --name web11 -p 8082:80 monsite1.1
Unable to find image 'monsite1.1:latest' locally
docker: Error response from daemon: pull access denied for monsite1.1, repository does not exist or may require 'dock
er login'

Run 'docker run --help' for more information
darkdiva@localhost:~/lab05$ docker run -d --name web11 -p 8082:80 monsite:1.1
d9721169ddf97851ab1d1cf7fedd5e610e03718329898f305519c2fcdb7a3b4f
```

Figure 24. Build of monsite:1.1 with the CACHED step, and the list of the two images.

```
darkdiva@localhost:~/lab05$ curl http://localhost:8082 | grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   213    0     0  18158      0 --:--:-- --:--:-- --:--:-- 19363
<p>Version 1.1 - service par nginx dans un conteneur.</p>
darkdiva@localhost:~/lab05$ curl -i http://localhost:8082/inconnu | head -1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   175  100   175    0     0  24329      0 --:--:-- --:--:-- --:--:-- 25000
HTTP/1.1 404 Not Found
darkdiva@localhost:~/lab05$ docker rm -f web11
web11
darkdiva@localhost:~/lab05$
```

Figure 25. Version 1.1 on port 8082: the new text and the 404 page.

4. Part 2: publishing the image in a registry

4.1 A registry with a volume. I started the registry with `docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2`. The registry stores its data in `/var/lib/registry`, and the option `-v` mounts the named volume `registry-data` there. `docker volume ls` lists it (the other volume is an older one that I do not use).

```
darkdiva@localhost:~/lab05$ docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2
bdc5698aa05c1852a33ae287812b88135277d92e27187e71147b7f34e2079763
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                    PORTS
                     NAMES
bdc5698aa05c   registry:2    "/entrypoint.sh /etc…"   9 seconds ago    Up 8 seconds              0.0.0.0:5000->5000/t
cp, [::]:5000->5000/tcp   registry
71b75cfd0d6c   monsite:1.0   "/docker-entrypoint.…"   40 minutes ago   Up 40 minutes (healthy)   0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp       web
darkdiva@localhost:~/lab05$ docker volume ls
DRIVER    VOLUME NAME
local     e2e3400296ea1b44faf7ea794b02652f036721fec81454829c6128968827824c
local     registry-data
darkdiva@localhost:~/lab05$
```

Figure 26. The registry container on port 5000 and the volume registry-data.

4.2 Tagging and pushing. The full name of an image tells Docker where to push it: registry, repository and tag. I tagged both versions as `localhost:5000/monsite:1.0` and `localhost:5000/monsite:1.1` and pushed them. In the push of 1.1, only two layers are sent (`2b38bf982f1a` and `3c94610ded0c`); all the others say “Layer already exists”, because 1.1 shares them with 1.0.

```
darkdiva@localhost:~/lab05$ docker tag monsite:1.0 localhost:5000/monsite:1.0
darkdiva@localhost:~/lab05$ docker tag monsite:1.1 localhost:5000/monsite:1.1
```

Figure 27. Tagging both versions with the address of the registry.

```
darkdiva@localhost:~/lab05$ docker push localhost:5000/monsite:1.0
The push refers to repository [localhost:5000/monsite]
44136fa355b3: Pushed
7b229517ed22: Pushed
9f0afb44f10e: Pushed
09c35a8bd1e0: Pushed
f19a2d3b6951: Pushed
885725a63a7c: Pushed
e125989d16f0: Pushed
81b64ae234ef: Pushed
bb658a254196: Pushed
5b8cb54cfa40: Pushed
2ad3a8567f12: Pushed
06bcd4f2ae3f: Pushed
1.0: digest: sha256:2a2565c044ee693d923024a7a09c7c7d28e167302982e837f7371d0525820ad8 size: 856
darkdiva@localhost:~/lab05$ docker push localhost:5000/monsite:1.1
The push refers to repository [localhost:5000/monsite]
2b38bf982f1a: Pushed
7b229517ed22: Layer already exists
f19a2d3b6951: Layer already exists
bb658a254196: Layer already exists
5b8cb54cfa40: Layer already exists
2ad3a8567f12: Layer already exists
9f0afb44f10e: Layer already exists
44136fa355b3: Already exists
885725a63a7c: Layer already exists
e125989d16f0: Layer already exists
81b64ae234ef: Layer already exists
3c94610ded0c: Pushed
1.1: digest: sha256:765f8c3bb1852b6d71d065833cce858fd0afa0d5a9cf9f1c6f8fa4ba8e8de905 size: 856
darkdiva@localhost:~/lab05$
```

Figure 28. Pushing 1.0 (all layers) and then 1.1 (only the changed layers).

> **Problem I ran into**
>
> My first push gave `tag does not exist: localhost:5000/monsite1.0:latest`. I had typed `monsite1.0` without the colon, so Docker read `monsite1.0` as the name of a repository and added the default tag `latest`, which I never created. The two versions of the name matter: `monsite:1.0` is the repository monsite with the tag 1.0.

```
darkdiva@localhost:~/lab05$ curl http://localhost:8082 | grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   213    0     0  18158      0 --:--:-- --:--:-- --:--:-- 19363
<p>Version 1.1 - service par nginx dans un conteneur.</p>
darkdiva@localhost:~/lab05$ curl -i http://localhost:8082/inconnu | head -1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   175  100   175    0     0  24329      0 --:--:-- --:--:-- --:--:-- 25000
HTTP/1.1 404 Not Found
darkdiva@localhost:~/lab05$ docker rm -f web11
web11
darkdiva@localhost:~/lab05$ docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2
bdc5698aa05c1852a33ae287812b88135277d92e27187e71147b7f34e2079763
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED          STATUS                    PORTS
                    NAMES
bdc5698aa05c   registry:2    "/entrypoint.sh /etc…"   9 seconds ago    Up 8 seconds              0.0.0.0:5000->5000/t
cp, [::]:5000->5000/tcp   registry
71b75cfd0d6c   monsite:1.0   "/docker-entrypoint.…"   40 minutes ago   Up 40 minutes (healthy)   0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp       web
darkdiva@localhost:~/lab05$ docker volume ls
DRIVER    VOLUME NAME
local     e2e3400296ea1b44faf7ea794b02652f036721fec81454829c6128968827824c
local     registry-data
darkdiva@localhost:~/lab05$
darkdiva@localhost:~/lab05$
darkdiva@localhost:~/lab05$
darkdiva@localhost:~/lab05$
darkdiva@localhost:~/lab05$ docker tag monsite:1.0 localhost:5000/monsite:1.0
darkdiva@localhost:~/lab05$ docker tag monsite:1.1 localhost:5000/monsite:1.1
darkdiva@localhost:~/lab05$ docker push localhost:5000/monsite1.0
Using default tag: latest
The push refers to repository [localhost:5000/monsite1.0]
tag does not exist: localhost:5000/monsite1.0:latest
darkdiva@localhost:~/lab05$ docker push localhost:5000/monsite:1.0
The push refers to repository [localhost:5000/monsite]
44136fa355b3: Pushed
7b229517ed22: Pushed
9f0afb44f10e: Pushed
09c35a8bd1e0: Pushed
f19a2d3b6951: Pushed
885725a63a7c: Pushed
```

Figure 29. The push with the missing colon, then the correct push starting.

4.3 Catalogue and tags. The registry has an HTTP API, so I queried it with `curl`. The catalogue lists `monsite`, and the list of tags of `monsite` shows `1.0` and `1.1`.

```
darkdiva@localhost:~/lab05$ curl http://localhost:5000/v2/_catalog
{"repositories":["monsite"]}
darkdiva@localhost:~/lab05$ curl http://localhost:5000/v2/monsite/tags/list
{"name":"monsite","tags":["1.0","1.1"]}
darkdiva@localhost:~/lab05$
```

Figure 30. The catalogue of the registry and the tags of monsite.

4.4 Proof that the volume keeps the data. I deleted the registry container with `docker rm -f registry`, checked with `docker ps` that it was gone, and created it again with the same `-v` option. Without pushing anything, the catalogue and the tags are still there. It is also a sign that the registry of Lab 04, which had no volume, is gone: its `hello` image is not in this catalogue.

```
darkdiva@localhost:~/lab05$ docker rm -f registry
registry
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE         COMMAND                  CREATED       STATUS                 PORTS
               NAMES
71b75cfd0d6c   monsite:1.0   "/docker-entrypoint.…"   2 hours ago   Up 2 hours (healthy)   0.0.0.0:8080->80/tcp, [::]
:8080->80/tcp   web
darkdiva@localhost:~/lab05$ docker run -d -p 5000:5000 --name registry -v registry-data:/var/lib/registry registry:2
dcd8cce67845b3d03d3a1565ae5b5ae5dec89568a797fd0f8931ff6af201fd1d
darkdiva@localhost:~/lab05$ curl http://localhost:5000/v2/_catalog
{"repositories":["monsite"]}
darkdiva@localhost:~/lab05$ curl http://localhost:5000/v2/monsite/tags/list
{"name":"monsite","tags":["1.0","1.1"]}
darkdiva@localhost:~/lab05$
```

Figure 31. The registry deleted and recreated: the images are still published.

4.5 Redeploying from the registry only. I deleted the web container and the local images of `monsite`, so that Docker could only get the site from the registry.

> **Problem I ran into**
>
> After the deletion, `curl http://localhost:8081` answered `Could not connect to server`. `docker ps -a` showed that there was simply no `site` container: I had not started it yet. `docker images | grep monsite` also still listed `localhost:5000/monsite:1.1`. I removed that image too, because a leftover local copy would make Docker use it instead of pulling from the registry.

```
darkdiva@localhost:~/lab05$ docker rm -f web
web
darkdiva@localhost:~/lab05$ docker rmi monsite:1.0 monsite:1.1 localhost:5000/monsite:1.0
Untagged: monsite:1.0
Untagged: monsite:1.1
Untagged: localhost:5000/monsite:1.0
Deleted: sha256:2a2565c044ee693d923024a7a09c7c7d28e167302982e837f7371d0525820ad8
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED         STATUS         PORTS
            NAMES
dcd8cce67845   registry:2   "/entrypoint.sh /etc…"   3 minutes ago   Up 3 minutes   0.0.0.0:5000->5000/tcp, [::]:5000
->5000/tcp   registry
darkdiva@localhost:~/lab05$ curl http://localhost:8081 |grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
curl: (7) Failed to connect to localhost port 8081 after 1 ms: Could not connect to server
darkdiva@localhost:~/lab05$ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                     PORTS
                      NAMES
dcd8cce67845   registry:2    "/entrypoint.sh /etc…"   8 minutes ago   Up 8 minutes               0.0.0.0:5000->5000/tcp
, [::]:5000->5000/tcp   registry
f7795f10d0ab   hello-world   "/hello"                 2 weeks ago     Exited (0) 2 weeks ago
                      naughty_borg
darkdiva@localhost:~/lab05$ docker logs site
Error response from daemon: No such container: site
darkdiva@localhost:~/lab05$ docker images | grep monsite
localhost:5000/monsite:1.1                 765f8c3bb185       91.3MB         26.1MB
```

Figure 32. No container on port 8081, and a leftover local image.

Then `docker run -d --name site -p 8081:80 localhost:5000/monsite:1.0` printed “Unable to find image locally” and pulled the layers from my registry. The container is `healthy` and `curl` shows **Version 1.0**.

```
darkdiva@localhost:~/lab05$ docker rmi localhost:5000/monsite:1.1
Untagged: localhost:5000/monsite:1.1
Deleted: sha256:765f8c3bb1852b6d71d065833cce858fd0afa0d5a9cf9f1c6f8fa4ba8e8de905
darkdiva@localhost:~/lab05$ docker images | grep monsite
darkdiva@localhost:~/lab05$ docker run -d --name site -p 8081:80 localhost:5000/monsite:1.0
Unable to find image 'localhost:5000/monsite:1.0' locally
1.0: Pulling from monsite
06bcd4f2ae3f: Pull complete
e125989d16f0: Already exists
7b229517ed22: Already exists
9f0afb44f10e: Already exists
f19a2d3b6951: Already exists
885725a63a7c: Already exists
bb658a254196: Already exists
81b64ae234ef: Already exists
5b8cb54cfa40: Already exists
2ad3a8567f12: Pull complete
44136fa355b3: Already exists
09c35a8bd1e0: Download complete
Digest: sha256:2a2565c044ee693d923024a7a09c7c7d28e167302982e837f7371d0525820ad8
Status: Downloaded newer image for localhost:5000/monsite:1.0
e1b2039994374b464ab09b59e306a6b10bdc1d4749d938937159904cc0c1b106
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS                    PORTS
                                  NAMES
e1b203999437   localhost:5000/monsite:1.0   "/docker-entrypoint.…"   43 seconds ago   Up 42 seconds (healthy)   0.0.0
.0:8081->80/tcp, [::]:8081->80/tcp       site
dcd8cce67845   registry:2                     "/entrypoint.sh /etc…"   15 minutes ago   Up 15 minutes             0.0.0
.0:5000->5000/tcp, [::]:5000->5000/tcp   registry
darkdiva@localhost:~/lab05$ curll http://localhost:8081 | grep Version
bash: curll: command not found...
Similar command is: 'curl'
darkdiva@localhost:~/lab05$ curl http://localhost:8081 | grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   213    0     0  66541      0 --:--:-- --:--:-- --:--:-- 71000
<p>Version 1.0 - service par nginx dans un conteneur.</p>
darkdiva@localhost:~/lab05$ docker rm -f sie
darkdiva@localhost:~/lab05$ docker rm -f site
site
```

Figure 33. Version 1.0 started from the registry on port 8081.

4.6 Update and rollback. To update the site from 1.0 to 1.1, I removed the container and ran the same command with another tag: `localhost:5000/monsite:1.1`. Only two layers were downloaded (the others were already on the VM) and `curl` shows **Version 1.1**. For the rollback, I removed the container again and ran the image with the tag 1.0. This time nothing is downloaded, and `time` shows **0.475 seconds**. The page says Version 1.0 again, and `docker ps` shows the container healthy.

```
darkdiva@localhost:~/lab05$ docker run -d --name site -p 8081:80 localhost:5000/monsite:1.1
Unable to find image 'localhost:5000/monsite:1.1' locally
1.1: Pulling from monsite
3c94610ded0c: Pull complete
e125989d16f0: Already exists
81b64ae234ef: Already exists
bb658a254196: Already exists
5b8cb54cfa40: Already exists
2ad3a8567f12: Already exists
885725a63a7c: Already exists
7b229517ed22: Already exists
9f0afb44f10e: Already exists
f19a2d3b6951: Already exists
44136fa355b3: Already exists
2b38bf982f1a: Download complete
Digest: sha256:765f8c3bb1852b6d71d065833cce858fd0afa0d5a9cf9f1c6f8fa4ba8e8de905
Status: Downloaded newer image for localhost:5000/monsite:1.1
299b7d7dc9e34a9a229bb0f45654f5d29e199e61c8f3eb9e9596354f314a04f6
darkdiva@localhost:~/lab05$ curl http://localhost:8081 | grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   213    0     0  78424      0 --:--:-- --:--:-- --:--:--  104k
<p>Version 1.1 - service par nginx dans un conteneur.</p>
darkdiva@localhost:~/lab05$ docker rm -f site
site
darkdiva@localhost:~/lab05$ time docker run -d --name site -p 8081:80 localhost:5000/monsite:1.0
efa10248ce0ff02f5e8cbee936302e2319d30a026c63d218926e8b26624d62ad

real    0m0.475s
user    0m0.004s
sys     0m0.027s
darkdiva@localhost:~/lab05$ curl http://localhost:8081 | grep Version
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   213    0     0  57043      0 --:--:-- --:--:-- --:--:-- 71000
<p>Version 1.0 - service par nginx dans un conteneur.</p>
```

Figure 34. Update to 1.1 (two layers pulled) and rollback to 1.0 (no download, 0.475 s).

```
darkdiva@localhost:~/lab05$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED              STATUS
   PORTS                                          NAMES
efa10248ce0f   localhost:5000/monsite:1.0   "/docker-entrypoint.…"   About a minute ago   Up About a minute (healthy)
   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp        site
dcd8cce67845   registry:2                   "/entrypoint.sh /etc…"   25 minutes ago       Up 25 minutes
   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp    registry
darkdiva@localhost:~/lab05$
```

Figure 35. The site and the registry running after the rollback.

4.7 Bonus: Docker Hub. I created a personal access token with the Read & Write permission in my Docker Hub account and used it as the password for `docker login -u darkdiva`. Then I tagged `localhost:5000/monsite:1.1` as `darkdiva/monsite:1.1` and pushed it. Most layers say “Mounted from library/nginx”: Docker Hub already had the nginx layers, so only three layers were really sent. The page of my repositories shows `darkdiva/monsite`. I logged out afterwards with `docker logout`.

```
darkdiva@localhost:~/lab05$ docker login -u darkdiva

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/home/darkdiva/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
darkdiva@localhost:~/lab05$ docker tag localhost:5000/monsite:1.1 darkdiva/monsite:1.1
darkdiva@localhost:~/lab05$ docker push darkdiva/monsite:1.1
The push refers to repository [docker.io/darkdiva/monsite]
44136fa355b3: Mounted from darkdiva/hello
7b229517ed22: Mounted from library/nginx
9f0afb44f10e: Mounted from library/nginx
2b38bf982f1a: Pushed
f19a2d3b6951: Mounted from library/nginx
885725a63a7c: Mounted from library/nginx
e125989d16f0: Mounted from library/nginx
81b64ae234ef: Mounted from library/nginx
bb658a254196: Mounted from library/nginx
5b8cb54cfa40: Mounted from library/nginx
2ad3a8567f12: Pushed
3c94610ded0c: Pushed
1.1: digest: sha256:765f8c3bb1852b6d71d065833cce858fd0afa0d5a9cf9f1c6f8fa4ba8e8de905 size: 856
darkdiva@localhost:~/lab05$
```

Figure 36. Login with the token (not shown), tag and push of darkdiva/monsite:1.1.

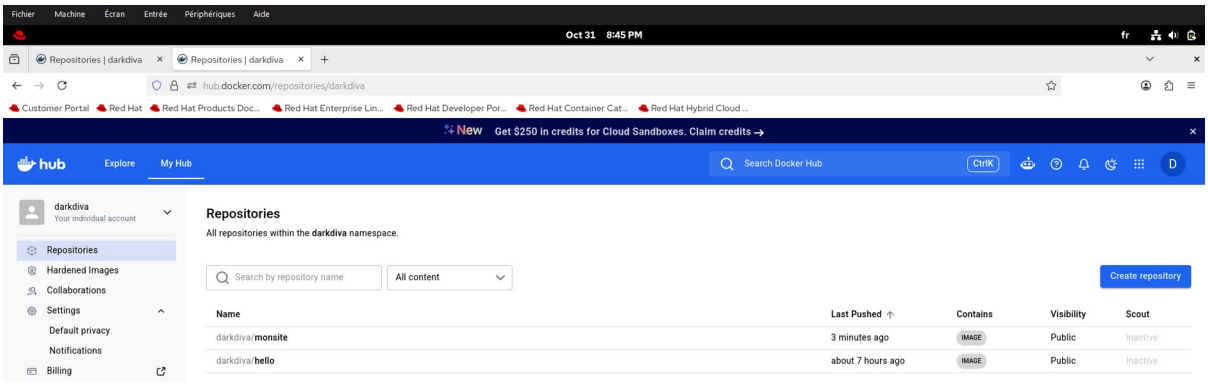


Figure 37. My repositories on Docker Hub: monsite has just been pushed.

```
darkdiva@localhost:~/lab05$ docker logout
Removing login credentials for https://index.docker.io/v1/
darkdiva@localhost:~/lab05$
```

Figure 38. docker logout removes the saved credentials.

5. Cleaning up

Before the cleanup, `docker system df` showed 3 containers and 15 images. I removed the containers `site` and `registry` with `docker rm -f`, and ran `docker image prune`. My first `prune` did not run, because at the confirmation prompt I typed another command instead of `y`; the second one worked. It freed 0 B, because I had no dangling images. The containers went from 3 to 1 (an old exited container remains) and the active images from 3 to 1. Most of the 3.2 GB of reclaimable space are unused base images, which `prune` does not touch. The volumes (26 MB) are now unused; I kept `registry-data` until the end of the lab, because it holds my published images.

```
darkdiva@localhost:~/lab05$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          15        3         3.426GB   3.184GB (92%)
Containers      3         2         8.192kB   0B (0%)
Local Volumes   2         1         26.12MB   31.63kB (0%)
Build Cache     55        0         2.402GB   169.4MB
darkdiva@localhost:~/lab05$ docker rm -f site registry
site
registry
darkdiva@localhost:~/lab05$ docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] docker system df
darkdiva@localhost:~/lab05$ y
bash: y: command not found...
darkdiva@localhost:~/lab05$ docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
darkdiva@localhost:~/lab05$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          15        1         3.426GB   3.22GB (93%)
Containers      1         0         0B        0B
Local Volumes   2         0         26.12MB   26.12MB (100%)
Build Cache     55        0         2.402GB   169.4MB
darkdiva@localhost:~/lab05$
```

Figure 39. docker system df before and after removing the containers, and docker image prune.

6. Answers to the questions

Why does my Dockerfile not need a CMD? Because the base image already defines how to start nginx. `docker inspect` shows the entrypoint `/docker-entrypoint.sh` and the command `nginx -g daemon off;`. An image built with `FROM` inherits them as long as it does not define its own. My Dockerfile only adds files and metadata (labels, port, health check), so nginx starts in the foreground exactly as it should. Writing my own `CMD` would only repeat what is already there.

What is the difference between EXPOSE 80 in the Dockerfile and -p 8080:80 in docker run? `EXPOSE 80` only documents the port the application listens on. It is metadata stored in the image and it opens nothing. `-p 8080:80` really publishes the port: it connects port 8080 of the VM to port 80 of the container, so the site can be reached from outside. Without `-p`, the site is not reachable from the VM even with `EXPOSE`, and with `-p` it works even if the Dockerfile has no `EXPOSE`.

In localhost:5000/monsite:1.1, what does each part mean? `localhost:5000` is the registry (the host and the port where it listens), `monsite` is the repository (the name of the image) and `1.1` is the tag (the version). When there is no registry in the name, Docker assumes Docker Hub, which is why the registry address must be part of the name to push to my own registry.

Why does the registry need a volume, and what would happen without it? The registry keeps its images in `/var/lib/registry`. Without a volume, that folder is in the writable layer of the container, which is deleted together with the container. If the registry container were removed, recreated or replaced, the registry would come back empty: no catalogue, no tags, and no way to redeploy the site until I built and pushed everything again. With the volume `registry-data`, I deleted and recreated the container and the catalogue and the tags were identical.

How do I roll back from 1.1 to 1.0, and why is it so fast? I remove the container and start it again with the same command, changing only the tag: `localhost:5000/monsite:1.0` instead of `1.1`. It is fast because the image 1.0 is still stored on the VM, so nothing is downloaded, and because the two versions share all their layers except the last ones. Docker only has to create a container from an image it already has: it took

0.475 seconds. Even when an image must be pulled, only the layers that are missing are downloaded, as the update to 1.1 showed (two layers).

7. Summary

Task	Result	Comment
Dockerfile with a pinned Alpine nginx, labels, EXPOSE and HEALTHCHECK	Done	nginx:1.30.6-alpine; docker ps shows healthy
.dockerignore	Done	notes-perso.txt and .git are not in the image
Site on port 8080, custom 404 page, logs	Done	default.conf had three mistakes to fix
Site opened from Windows	Done	Port forwarding rule in VirtualBox
Version 1.1 and build with the cache	Done	Only COPY site/ was rebuilt
Registry with a volume	Done	Images still there after deleting the container
Publishing 1.0 and 1.1, catalogue and tags	Done	Tags 1.0 and 1.1 listed with curl
Redeploy from the registry, update, rollback	Done	Rollback in 0.475 s
Bonus: Docker Hub	Done	darkdiva/monsite:1.1; token not shown
Cleanup	Done	Containers removed; prune freed 0 B

8. What I learned

The biggest lesson was how much the order of the instructions matters. Because the site pages and the version label are at the end of the Dockerfile, building version 1.1 reused everything above them from the cache, and the registry only had to store and send two new layers. The same layers made the rollback instant. I also understood that a HEALTHCHECK is more useful than it looks: `docker ps` told me at once that the container was really serving the page and not only running.

The registry taught me that a container's data disappears with the container unless it is in a volume, and that the name of an image is also its address. A colon missing in `monsite:1.0` was enough to make Docker look for a different repository.

Most of my errors came from details typed by hand: a semicolon and a space in `default.conf`, a colon in an image name, and a build started from the wrong folder. I learned to read the error message first, to look at the hidden characters with `cat -A`, and to remember that a file copied into an image only changes when the image is rebuilt.